

Aula 02 — O JavaScript que profissionais escrevem

Curso de Formação de Programação Aplicada · Módulo 1 · FPA Labs

Duração: 2h · **Pré-requisito:** Aula 01

Objetivos de aprendizagem

Ao final desta aula o aluno deve conseguir:

1. Escolher entre `const` e `let` com critério (e nunca mais usar `var`).
2. Escrever funções em formato de arrow function.
3. Transformar, filtrar e resumir listas com `map`, `filter` e `reduce`.
4. Usar desestruturação e spread para trabalhar com objetos e arrays.
5. Explicar por que **não alteramos** os dados originais (imutabilidade) — conceito que será fundamental no React, na aula 08.

Plano de aula (2h)

Tempo	Bloco	Conteúdo
0:00–0:08	Aquecimento	Correção do exercício 4 da aula 01 (conversor pelo terminal)
0:08–0:20	Conceito	<code>const</code> / <code>let</code> · arrow functions · como ler um método
0:20–0:50	Código ao vivo	<code>map</code> → <code>filter</code> → <code>reduce</code> , com a ficha de cada um antes de usar
0:50–1:00	Intervalo	
1:00–1:20	Código ao vivo	Encadeamento · desestruturação · spread · imutabilidade
1:20–1:50	Mão na massa	Exercícios 1 a 3 (o 4 fica de tarefa)
1:50–2:00	Fechamento	Recapitulação · onde consultar métodos · prévia da aula 03

⚠ **Aula mais densa do módulo.** São muitos métodos novos, e o material foi escrito para que nenhum apareça sem explicação. Isso tem um custo de tempo — por isso o plano acima tem **dois** blocos de código ao vivo e o exercício 4 já sai como tarefa.

O que detalhar (sem pressa): `map`, `filter`, `reduce` e imutabilidade. **O que passar rápido, dizendo "isto fica de consulta":** `find`, `some`, `every`, `includes`, `join`, `forEach` e os métodos de formatação (`toFixed`, `padEnd`, `toLocaleString`, `Math`).

Diga à turma, em voz alta: **"não precisa anotar, está tudo no material"**. Aluno iniciante tenta copiar tudo e para de acompanhar o raciocínio.

1. `const`, `let` e o `var` aposentado

```
const nome = "Maria"; // não vou trocar o valor desta caixa
let contador = 0;      // vou trocar
var qualquerCoisa = 1; // ❌ não use mais
```

A regra do curso é simples: comece sempre com `const`. Só troque para `let` quando o JavaScript reclamar que você está tentando reatribuir. Isso faz o leitor do código saber, num relance, o que muda e o que não muda.

A pegadinha mais comum

```
const lista = [1, 2, 3];
lista.push(4); // ✅ funciona! lista agora é [1, 2, 3, 4]
lista = [9, 9, 9]; // ❌ TypeError: Assignment to constant variable
```

`const` **não** significa "conteúdo imutável". Significa: "esta variável sempre vai apontar para esta mesma caixa". Você não pode trocar a caixa, mas pode mexer no que tem dentro dela.

2. Arrow functions

Três formas de escrever a mesma função:

```
// 1. Clássica
function dobrar(n) {
  return n * 2;
}

// 2. Arrow com corpo
const dobrar = (n) => {
  return n * 2;
};

// 3. Arrow curta – quando o corpo é UMA expressão, o return é implícito
const dobrar = (n) => n * 2;
```

A forma 3 é a que mais aparece em código profissional, principalmente dentro de `map` e `filter`.

⚠ **Cuidado com o retorno implícito de objetos.** Isto não funciona como você espera:

```
const criar = (nome) => { nome: nome }; // ❌ retorna undefined
const criar = (nome) => ({ nome: nome }); // ✅ os parênteses resolvem
```

Sem os parênteses, o JavaScript entende as chaves como o *corpo* da função, não como um objeto.

3. Antes: como ler um método

A partir de agora vamos usar muitos **métodos nativos** — funções que já vêm prontas dentro do JavaScript. Antes da lista, uma habilidade que vale mais que decorar qualquer um deles:

```
vendas.map( (venda) => venda.produto )
```

↑ ↑ ↑

| | | o que você ENTREGA para ele (aqui, uma função)

| | | o NOME do método

| | | em QUEM você está chamando (um array)

Três perguntas para fazer sobre qualquer método novo:

Pergunta	Por que importa
O que ele devolve?	um valor novo? nada (<code>undefined</code>)?
Ele altera o original?	isso vai mudar tudo quando chegarmos no React
O que acontece se não achar nada?	<code>undefined</code> , <code>-1</code> , <code>[]</code> ? Cada método responde diferente

Faça essas três perguntas para cada método desta aula. Elas explicam quase todos os bugs que você vai encontrar nas próximas semanas.

Referência do curso: [recursos/METODOS-NATIVOS.md](#) tem todos os métodos usados no módulo, no formato *o que faz · quando usar · exemplo · pegadinha*. Use como dicionário, não como leitura corrida.

4. Trabalhando com listas: o trio essencial

Este é o coração da aula. Praticamente todo código de back-end é: *pegar uma lista de dados, transformar, filtrar e resumir*.

Nossos dados de exemplo:

```
const vendas = [
  { produto: "Notebook", categoria: "eletronicos", valor: 3500, quantidade: 2 },
  { produto: "Mouse", categoria: "eletronicos", valor: 80, quantidade: 10 },
  { produto: "Cadeira", categoria: "moveis", valor: 900, quantidade: 3 },
  { produto: "Mesa", categoria: "moveis", valor: 1200, quantidade: 1 },
  { produto: "Teclado", categoria: "eletronicos", valor: 250, quantidade: 5 },
];
```

map — transformar cada item

O que faz	transforma cada item da lista
Quando usar	"quero a mesma lista, mas com cada item virando outra coisa"
Devolve	um array novo , do mesmo tamanho
Altera o original?	✗ não
Lista vazia?	devolve <code>[]</code>

"Para cada item, me devolva outra coisa." **Entram 5, saem 5.**

```
const nomes = vendas.map((venda) => venda.produto);
// ["Notebook", "Mouse", "Cadeira", "Mesa", "Teclado"]

const totais = vendas.map((venda) => ({
  produto: venda.produto,
  total: venda.valor * venda.quantidade,
}));
```

filter — escolher itens

O que faz mantém só os itens aprovados pela função

Quando usar "quero só alguns itens desta lista"

Devolve um array **novo**, com 0 a N itens

Altera o original? ❌ não

Nada bateu? devolve `[]` (array vazio) — **nunca** `undefined`

"Para cada item, ele fica ou sai?" **Entram 5, saem 0 a 5.**

```
const eletronicos = vendas.filter((venda) => venda.categoria === "eletronicos");
// 3 itens
```

A função dentro do `filter` responde uma pergunta de sim/não. `true` mantém, `false` descarta.

Truque que você vai usar muito: remover um item sem alterar o array original.

```
lista.filter((item) => item.id !== 3); // remove por valor
lista.filter((item, indice) => indice !== 2); // remove por posição
```

reduce — resumir tudo em um valor

O que faz percorre a lista acumulando, e devolve **um** valor

Quando usar somar, contar, agrupar, achar o maior — qualquer coisa que resuma

Devolve um valor só (número, objeto, array — o que você quiser)

Altera o original? ❌ não

Atenção recebe **dois** argumentos: a função **e** o valor inicial

"Vá acumulando." **Entram 5, sai 1.**

```
const faturamento = vendas.reduce((acumulador, venda) => {
  return acumulador + venda.valor * venda.quantidade;
}, 0);
//                                     ↑ valor inicial
```

O `reduce` assusta no começo. Destrinche-o na lousa, passo a passo:

Rodada	acumulador chega como	item	devolve
1	0 (o valor inicial)	Notebook	7000
2	7000	Mouse	7800
3	7800	Cadeira	10500
4	10500	Mesa	11700
5	11700	Teclado	12950

O valor devolvido em uma rodada é o `acumulador` da próxima. No fim, sobra `12950`.

Nunca esqueça o valor inicial (o `, 0` no final). Sem ele, o `reduce` usa o primeiro item da lista como acumulador — e em lista de objetos isso gera resultados bizarros.

Encadeando: o poder de verdade

Como cada método devolve um novo array, dá para ligar um no outro:

```
const faturamentoEletronicos = vendas
  .filter((venda) => venda.categoria === "eletronicos")
  .map((venda) => venda.valor * venda.quantidade)
  .reduce((total, valor) => total + valor, 0);
```

Leia em voz alta: “das vendas, **fique só** com eletrônicos, **transforme** cada uma no total dela, e **some tudo**”. O código virou uma frase.

Compare com o `for` equivalente:

```
let faturamentoEletronicos = 0;
for (let i = 0; i < vendas.length; i++) {
  if (vendas[i].categoria === "eletronicos") {
    faturamentoEletronicos += vendas[i].valor * vendas[i].quantidade;
  }
}
```

Funciona igual. Mas o primeiro diz **o que** você quer; o segundo diz **como** fazer, passo a passo. Em código profissional, o primeiro vence quase sempre.

Os outros que você vai usar toda hora

Não decore: entenda o que cada um **devolve** e o que acontece **quando não acha nada**.

`.FIND()` E `.FINDINDEX()`

```
vendas.find((v) => v.produto === "Mouse");    // o objeto, ou undefined
vendas.findIndex((v) => v.produto === "Mesa"); // a posição (3), ou -1
```

A diferença que mais confunde — `find` × `filter`:

	devolve	quando não acha
<code>.find()</code>	o objeto	<code>undefined</code>
<code>.filter()</code>	um array	<code>[]</code> (vazio)

Espera **um** resultado → `find`. Espera **vários** → `filter`.

⚠ **Pegadinha do `findIndex`**: ele devolve `-1` quando não acha. Sempre teste com `if (indice === -1)`, nunca com `if (!indice)` — porque a posição `0` também é "falsa" em JavaScript, e o primeiro item da lista viraria um bug silencioso.

`.SOME()` E `.EVERY()`

```
vendas.some((v) => v.valor > 3000);    // true - existe PELO MENOS UM?
vendas.every((v) => v.valor > 50);     // true - TODOS atendem?
```

Devolvem `true` ou `false`. Uso real: verificar duplicidade antes de criar um registro (`filmes.some((f) => f.titulo === novoTitulo)`) — você fará exatamente isso na aula 05.

`.INCLUDES()`

```
["baixa", "media", "alta"].includes("alta");    // true
["baixa", "media", "alta"].includes("urgente"); // false
```

Serve para arrays e para textos. Uso mais comum no curso: validar que um valor está na lista de permitidos.

Diferença para o `.some()`: `includes` compara **valores diretos**; `some` recebe uma **função** e serve para condições mais complexas.

.JOIN()

```
["a", "b", "c"].join(", ");    // "a, b, c"
erros.join(" · ");             // "título é obrigatório · ano inválido"
```

Transforma um array em texto. É o **oposto** do `.split()`, que transforma texto em array.

.SORT()

```
[...vendas].sort((a, b) => b.valor - a.valor); // do maior para o menor
```

⚠ **sort** altera o array original. Copie antes com `[...vendas]`, ou use a versão moderna `vendas.toSorted(...)`, que já devolve um array novo.

Como a função de comparação funciona: ela recebe dois itens e devolve um número.

Devolveu	Significa
negativo	<code>a</code> vem antes de <code>b</code>
zero	tanto faz
positivo	<code>b</code> vem antes de <code>a</code>

Por isso `a - b` ordena crescente e `b - a`, decrescente.

⚠ **Sem função de comparação, o `sort` ordena como texto:**

```
[10, 9, 100].sort()           // [10, 100, 9] 🗿
[10, 9, 100].sort((a, b) => a - b) // [9, 10, 100] ✅
```

Para ordenar **nomes em português**, use `.localeCompare()` — ele respeita acentos:

```
nomes.sort((a, b) => a.localeCompare(b)); // "Ana", "Ávila", "Bruno" ✅
```

Sem ele, o JavaScript compara pelo código numérico das letras, e "Á" acaba caindo depois do "Z".

.FOREACH()

```
lista.forEach((item) => console.log(item));
```

Executa a função para cada item, mas **não devolve nada**. Use quando só quer *fazer* algo (imprimir, por exemplo). Se você quer uma lista nova, use `.map()`.

⚠️ `forEach` não funciona com `await` — veremos isso na aula 03.

A tabela que você precisa guardar

Altera o array original 🛑	Devolve um array novo 🟢
<code>.push()</code> , <code>.pop()</code> , <code>.shift()</code> , <code>.unshift()</code>	<code>.map()</code> , <code>.filter()</code> , <code>.slice()</code>
<code>.splice()</code>	<code>.concat()</code> , <code>.toSorted()</code>
<code>.sort()</code> , <code>.reverse()</code>	spread <code>[...lista]</code>

Esta divisão parece detalhe hoje. Na aula 08 ela será a diferença entre a tela do React atualizar ou não.

5. Destruturização

Uma forma curta de tirar valores de dentro de objetos e arrays.

```
const usuario = { nome: "Maria", idade: 28, cidade: "São Paulo" };

// Sem destruturação
const nome = usuario.nome;
const idade = usuario.idade;

// Com destruturação
const { nome, idade } = usuario;

// Renomeando e com valor padrão
const { nome: nomeCompleto, pais = "Brasil" } = usuario;
```

Em arrays, vale a **posição**:

```
const [primeiro, segundo] = ["a", "b", "c"];
```

O uso mais frequente é em parâmetros de função:

```
// Em vez de:
function saudar(usuario) {
  return `Olá, ${usuario.nome}`;
}

// Escreva:
function saudar({ nome }) {
  return `Olá, ${nome}`;
}
```

Você vai ver isso o tempo todo no Express (`const { id } = req.params`) e no React (`function Card({ titulo })`). Vale dominar hoje.

6. Percorrendo objetos e formatando a saída

O relatório que vamos construir usa mais alguns métodos nativos. Vale conhecer cada um antes de vê-los no meio do código.

`Object.entries()` — o que destrava objetos

Objetos **não têm** `.map()`, `.filter()` nem `.sort()`. Esses métodos são de array.

```
const contagem = { drama: 2, acao: 1 };
contagem.map(...); // ❌ não existe
```

`Object.entries()` resolve isso transformando o objeto em um array de pares `[chave, valor]`:

```
Object.entries(contagem);
// [["drama", 2], ["acao", 1]] ← agora é array, e arrays têm map, filter, sort...
```

A família completa:

Método	Devolve	Exemplo com <code>{ nome: "Ana", idade: 28 }</code>
<code>Object.keys(obj)</code>	os nomes das propriedades	<code>["nome", "idade"]</code>
<code>Object.values(obj)</code>	os valores	<code>["Ana", 28]</code>
<code>Object.entries(obj)</code>	os pares	<code>[["nome", "Ana"], ["idade", 28]]</code>

Todas devolvem array. Uso prático muito comum: `Object.keys(obj).length === 0` responde "esse objeto está vazio?".

Percorrendo, com desestruturação de array:

```
for (const [categoria, total] of Object.entries(dados)) {  
  console.log(`${categoria}: ${total}`);  
}
```

`.toFixed()` — casas decimais

```
(3.14159).toFixed(2);    // "3.14"  
(10 / 3).toFixed(1);     // "3.3"  
(8).toFixed(2);         // "8.00"
```

⚠ Devolve texto, não número. Isso pega muita gente:

```
(3.14159).toFixed(2) + 1;    // "3.141" 🗨 concatenou  
Number((3.14159).toFixed(2)) + 1; // 4.14 ✅
```

Regra: use `toFixed` para **mostrar**, nunca para continuar calculando.

`.toLocaleString("pt-BR")` — formato brasileiro

```
(1234.5).toLocaleString("pt-BR");    // "1.234,5"  
  
(1234.5).toLocaleString("pt-BR", { style: "currency", currency: "BRL" });  
// "R$ 1.234,50"
```

Ponto no milhar, vírgula no decimal, símbolo da moeda — tudo resolvido sem você fazer conta nem manipular string.

`.padEnd()` e `.padStart()` — alinhando no terminal

```
"Ana".padEnd(12);           // "Ana           " (completa até 12 caracteres)  
"Bernardo".padEnd(12);     // "Bernardo     "  
"5".padStart(2, "0");      // "05"
```

É o que transforma isto:

```
Ana R$ 100  
Bernardo R$ 250
```

nisto:

Ana R\$ 100
Bernardo R\$ 250

⚠ Se o texto já for maior que o tamanho pedido, nada acontece — `padEnd` **não corta**.

A família `Math`

Função	O que faz	<code>Math.x(4.3)</code>	<code>Math.x(4.7)</code>
<code>Math.round</code>	arredonda normal	4	5
<code>Math.ceil</code>	arredonda sempre para cima	5	5
<code>Math.floor</code>	arredonda sempre para baixo	4	4

`Math.ceil` é o que você usa em paginação: com 5 itens e 2 por página, $5/2 = 2.5$, e você precisa de 3 páginas.

```
Math.max(3, 7, 2);           // 7
Math.max([3, 7, 2]);          // NaN 🤔 - não aceita array
Math.max(...[3, 7, 2]);       // 7 ✅ - spread "espalha" o array
Math.max(...lista.map((i) => i.id)); // o maior id - usaremos na aula 05
```

7. Spread e rest: os três pontinhos

O mesmo símbolo `...` faz duas coisas, dependendo de onde está.

Spread — "espalha o conteúdo aqui"

```
const numeros = [1, 2, 3];
const maisNumeros = [...numeros, 4, 5]; // [1, 2, 3, 4, 5]

const usuario = { nome: "Maria", idade: 28 };
const atualizado = { ...usuario, idade: 29 }; // copia tudo e sobreescreve idade
```

Esse padrão — **copiar e mudar uma coisa** — é o mais importante da aula. Você vai usá-lo em absolutamente todo lugar no React.

Rest — "junta o resto num array"

```
function somarTodos(...numeros) {  
  return numeros.reduce((total, n) => total + n, 0);  
}  
  
somarTodos(1, 2, 3, 4);    // 10  
  
const { nome, ...restoDoUsuario } = usuario;    // separa uma chave do resto
```

8. Imutabilidade: não mexa no original

Compare:

```
// ❌ MUTAÇÃO — altera o array original  
function adicionarItem(lista, item) {  
  lista.push(item);  
  return lista;  
}  
  
// ✅ IMUTÁVEL — devolve um array novo  
function adicionarItem(lista, item) {  
  return [...lista, item];  
}
```

Por que isso importa?

1. Ninguém é pego de surpresa. Se você passa uma lista para uma função, ela não deveria voltar diferente sem você saber. Bugs assim são difíceis de achar.
2. **O React só redesenha a tela quando enxerga um objeto novo.** Se você mutar o array, a tela simplesmente não atualiza — e este é o erro número 1 de quem está começando em React. Estamos plantando essa semente hoje, seis aulas antes.

Guarde os equivalentes imutáveis:

Em vez de (muta)	Use (não muta)
<code>lista.push(x)</code>	<code>[...lista, x]</code>
<code>lista.unshift(x)</code>	<code>[x, ...lista]</code>
<code>lista.splice(i, 1)</code>	<code>lista.filter((_, idx) => idx !== i)</code>
<code>obj.chave = valor</code>	<code>{ ...obj, chave: valor }</code>
<code>lista.sort(...)</code>	<code>[...lista].sort(...)</code>

9. Extras que salvam o dia

?. — optional chaining

O que faz: acessa a propriedade **só se** o objeto existir. Se não existir, devolve `undefined` em vez de quebrar o programa.

```
usuario.endereco.cidade;    // ✨ TypeError se endereco for undefined
usuario?.endereco?.cidade;   // undefined, sem quebrar nada
```

Leia como: "se existir, continua; se não, para aqui e devolve undefined".

Funciona também para chamar métodos que podem não existir:

```
corpo.erros?.join(", ");    // só chama join se erros existir
```

?? — nullish coalescing (valor padrão)

O que faz: usa o valor da direita **só se** o da esquerda for `null` ou `undefined`.

```
const limite = configuracao.limite ?? 10;
const prioridade = dados.prioridade ?? "media";
```

A diferença crucial para o `||` — e o motivo de o `??` existir:

```
0 || 10;    // 10 🚫 - zero é "falso", então o || troca
0 ?? 10;    // 0 ✅ - zero é um valor válido, o ?? mantém

"" || "padrão"; // "padrão"
"" ?? "padrão"; // ""
```

O `||` troca quando o valor é **falso** (`0`, `""`, `false`, `null`, `undefined`). O `??` troca só quando o valor **não existe** (`null` ou `undefined`).

Regra do curso: para valor padrão, prefira `??`. Nota `0` e texto vazio costumam ser respostas legítimas do usuário, e o `||` as descartaria silenciosamente.

JSON — o idioma entre front e back

```
const texto = JSON.stringify(usuario); // objeto → texto
const objeto = JSON.parse(texto);     // texto → objeto
```

JSON é **sempre texto**. É nesse formato que os dados viajam pela internet — por isso essas duas funções vão aparecer em toda requisição a partir da aula 03.

```
JSON.stringify(dados, null, 2); // com indentação de 2 espaços, para humanos lerem
```

⚠ `JSON.parse` **quebra** se o texto for inválido. Quando o texto vem de fora, proteja com `try/catch` (aula 03).

Ferramentas novas desta aula

Todas explicadas acima e detalhadas em [recursos/METODOS-NATIVOS.md](#).

Ferramenta	Em uma frase	Altera o original?
<code>.map()</code>	transforma cada item	✗
<code>.filter()</code>	mantém só alguns itens	✗
<code>.reduce()</code>	resume a lista em um valor	✗
<code>.find()</code>	o primeiro que bater (ou <code>undefined</code>)	✗
<code>.findIndex()</code>	a posição (ou <code>-1</code>)	✗
<code>.some()</code> / <code>.every()</code>	existe algum? / todos atendem?	✗
<code>.includes()</code>	esse valor está na lista?	✗
<code>.join()</code>	array vira texto	✗
<code>.forEach()</code>	faz algo com cada item, não devolve nada	✗
<code>.sort()</code>	ordena	🔴 sim — copie antes
<code>.toSorted()</code>	ordena devolvendo array novo	✗
<code>.localeCompare()</code>	compara textos respeitando acentos	✗
<code>Object.keys/values/entries()</code>	objeto vira array	✗
<code>.toFixed()</code>	casas decimais (devolve texto)	✗
<code>.toLocaleString("pt-BR")</code>	formato brasileiro / moeda	✗
<code>.padEnd()</code> / <code>.padStart()</code>	completa com espaços para alinhar	✗
<code>Math.round/ceil/floor/max/min</code>	arredondar e comparar números	✗
<code>?.</code>	acessa só se existir	—
<code>??</code>	valor padrão quando não existe	—
<code>JSON.stringify/parse</code>	objeto ↔ texto	—

Recapitulação em 3 frases

1. `map` transforma, `filter` seleciona, `reduce` resume — e eles se encadeiam como uma frase.
2. Desestruturação tira valores de dentro; `...` copia e espalha.
3. Nunca altere o dado original: copie, mude a cópia, devolva a cópia.

Tarefa para a próxima aula

Exercícios 1 a 4 de [exercicios/README.md](#).

Na próxima aula

Vamos entender o que acontece quando o código precisa **esperar** — ler um arquivo, chamar uma API. É o conceito que separa quem entende Node de quem apenas copia código: **assincronia**.
